

FINAL PROJECT STRUKTUR DATA

Topik 8 : Disaster Relief Distribution Network

1. Identitas Kelompok

Kelompok : 5

Kelas : Struktur Data dan OOP B

No	NRP	Nama Anggota
1	5027251005	Umar
2	5027251012	Gede Satya Putra Aryanta
3	5027251036	Jonathan Steven Tjahjaputra
4	5027251087	Muhammad Rifki Pribadi
5	5027251111	Azfaro Zid Ilmi

2. Deskripsi Masalah

Dalam situasi bencana alam, kecepatan dan efisiensi pengiriman bantuan logistik adalah kunci untuk menyelamatkan nyawa. Namun, akses infrastruktur (jalan) seringkali rusak atau terputus akibat bencana. Masalah yang harus diselesaikan oleh sistem ini meliputi:

- Bagaimana menentukan prioritas posko mana yang harus dibantu terlebih dahulu berdasarkan tingkat kerusakan dan kebutuhannya?
- Bagaimana mencari rute perjalanan tercepat menuju posko yang terdampak ketika beberapa ruas jalan tidak bisa dilewati?
- Bagaimana merancang jaringan distribusi yang paling hemat biaya operasional agar seluruh daerah terhubung namun dengan konsumsi bahan bakar kendaraan yang minimal?

Sistem ini digunakan sebagai simulasi penyelesaian masalah di atas secara komprehensif.

3. Dataset dan Atributnya

Data direpresentasikan menggunakan graph berbobot tidak berarah, dengan node sebagai lokasi distribusi dan edge sebagai jalur penghubung antar lokasi.

Setiap node memiliki atribut sebagai berikut:

- a. **id** - Kode unik yang digunakan untuk membedakan setiap lokasi, seperti G01 untuk gudang, P01 untuk posko, dan D01 untuk desa.
- b. **name** - Nama lokasi yang digunakan agar hasil pencarian rute lebih mudah dipahami oleh pengguna.
- c. **type** - Jenis lokasi yang menunjukkan peran node dalam proses distribusi, yaitu gudang, posko, atau desa.
- d. **population** - Jumlah penduduk pada suatu wilayah. Atribut ini dapat digunakan sebagai salah satu pertimbangan dalam menentukan prioritas distribusi bantuan.
- e. **criticalLevel** - Tingkat urgensi kebutuhan bantuan pada suatu lokasi. Semakin tinggi nilainya, semakin mendesak lokasi tersebut untuk ditangani.
- f. **logisticsNeeded** - Jumlah bantuan logistik yang dibutuhkan oleh suatu wilayah.
- g. **riskLevel** - Tingkat resiko akses menuju lokasi tersebut. Semakin tinggi nilainya, semakin beresiko lokasi tersebut untuk didatangi.

Setiap edge memiliki atribut sebagai berikut:

- a. **sourceId** - ID lokasi asal dari suatu jalur.
- b. **destinationId** - ID lokasi tujuan yang terhubung dengan lokasi asal.
- c. **distance** - Jarak antar-lokasi dalam satuan kilometer. Nilai ini berfungsi sebagai bobot edge dalam proses pencarian rute.

4. Struktur Data yang Digunakan

4.1 Graph dan Representasi Adjacency List

Rute distribusi bantuan logistik direpresentasikan menggunakan struktur data Graph Undirected Weighted. Graph bersifat undirected karena jalan dapat dilalui dua arah, dan weighted karena setiap jalur memiliki bobot berupa jarak tempuh.

- Node (vertex) merepresentasikan lokasi seperti Gudang Pusat, Posko, dan Desa.
- Edge merepresentasikan jalan yang menghubungkan dua lokasi.
- Weight pada edge merepresentasikan jarak tempuh antar lokasi dalam kilometer.
- Graph disimpan menggunakan Adjacency List dengan struktur Map<String, List<RouteEdge>>.

Adjacency List dipilih karena jaringan jalan pada dataset bersifat sparse, yaitu satu lokasi hanya terhubung ke beberapa lokasi lain di sekitarnya. Representasi ini lebih hemat memori dengan kompleksitas ruang $O(V + E)$ dibandingkan Adjacency Matrix yang membutuhkan $O(V^2)$.

4.2 Min-Heap sebagai Struktur Tree

Min-Heap merupakan struktur data Tree berbentuk Complete Binary Tree. Pada Min-Heap, elemen dengan prioritas terkecil secara konsep berada di root. Dalam project ini, Min-Heap diimplementasikan secara custom tanpa menggunakan PriorityQueue bawaan Java.

Min-Heap digunakan untuk mengelola prioritas distribusi bantuan. Skor kedaruratan dihitung menggunakan rumus berikut:

$$\text{Skor} = (\text{Kritis} \times 100) + (\text{Risiko} \times 40) + (\text{Kebutuhan} \times 5) + (\text{Populasi} / 100)$$

Semakin tinggi skor, semakin tinggi urgensi suatu lokasi. Karena struktur yang digunakan adalah Min-Heap, program membalik logika perbandingan pada class Task. Dengan demikian, lokasi dengan skor kedaruratan terbesar dianggap sebagai prioritas tertinggi dan dapat diekstrak lebih dahulu menggunakan operasi extractMin.

- insert
- extractMin
- heapifyUp
- heapifyDown
- isEmpty
- size

5. Algoritma yang Digunakan

5.1 Algoritma Dijkstra

Algoritma Dijkstra digunakan untuk fitur pencarian rute tercepat. Tujuan algoritma ini adalah menemukan rute dengan total jarak paling pendek dari satu titik awal, seperti Gudang atau Posko, menuju titik tujuan tertentu, seperti Posko atau Desa terdampak.

Dijkstra dipilih karena sesuai untuk menyelesaikan permasalahan Single-Source Shortest Path pada graph yang memiliki bobot positif. Dalam project ini, bobot edge berupa jarak fisik sehingga tidak mungkin bernilai negatif.

Dalam implementasi program, algoritma Dijkstra menggunakan MinHeap custom sebagai struktur antrean prioritas. MinHeap digunakan untuk mengambil node dengan jarak sementara paling kecil melalui operasi extractMin. Setelah itu, algoritma melakukan proses relaxation, yaitu memperbarui jarak ke node tetangga apabila ditemukan rute yang lebih pendek.

5.2 Algoritma Kruskal MST

Algoritma Kruskal digunakan untuk membentuk Jaringan Distribusi Minimum atau Minimum Spanning Tree. Tujuan algoritma ini adalah menghubungkan seluruh lokasi dengan total jarak sekecil mungkin tanpa membentuk siklus.

Kruskal sesuai digunakan karena graph dapat diproses melalui daftar edge. Semua edge aktif dikumpulkan dan diurutkan berdasarkan bobot dari yang terkecil hingga terbesar. Edge kemudian dievaluasi satu per satu. Edge akan dimasukkan ke dalam MST apabila tidak membentuk siklus. Pengecekan siklus dilakukan menggunakan struktur data tambahan Disjoint Set atau Union-Find.

Dijkstra lebih tepat digunakan ketika sistem membutuhkan rute terbaik dari satu lokasi asal ke satu lokasi tujuan tertentu, misalnya dari Gudang Pusat ke Desa terdampak. Sebaliknya, Kruskal MST lebih tepat digunakan ketika sistem ingin merancang jaringan distribusi minimum yang menghubungkan seluruh lokasi dengan total jarak paling kecil. Dengan

demikian, Dijkstra cocok untuk pengiriman bantuan secara spesifik, sedangkan Kruskal MST cocok untuk perencanaan jaringan distribusi secara keseluruhan.

5.3 Simulasi Jalan Rusak dan Konektivitas

Fitur simulasi jalan rusak dilakukan dengan mengubah status active pada edge. Jika status edge menjadi tidak aktif, algoritma Dijkstra dan Kruskal akan mengabaikan jalur tersebut. Selain itu, program menyediakan fitur cek konektivitas jaringan menggunakan traversal BFS agar dapat diketahui apakah seluruh lokasi masih terhubung melalui jalan aktif.

6. Design Decision Log

No	Keputusan Desain	Alasan
1	Memilih topik Disaster Relief Distribution Network	Topik ini cocok untuk penerapan Graph dan Tree karena ada lokasi, jalur distribusi, prioritas bantuan, dan kondisi jalan yang bisa berubah.
2	Menggunakan Graph untuk jaringan distribusi	Lokasi seperti gudang, posko, dan desa dapat direpresentasikan sebagai node, sedangkan jalan distribusi sebagai edge.
3	Menggunakan undirected weighted graph	Jalan antar lokasi diasumsikan bisa dilalui dua arah, dan setiap jalan memiliki bobot berupa jarak dalam kilometer.
4	Menggunakan adjacency list	Jumlah koneksi antar lokasi tidak terlalu padat, sehingga adjacency list lebih hemat memori daripada adjacency matrix.
5	Menggunakan Min-Heap sebagai struktur Tree	Min-Heap cocok untuk mengambil data prioritas secara cepat, terutama untuk menentukan lokasi yang harus dibantu lebih dulu.
6	Menggunakan skor gabungan untuk prioritas bantuan	Prioritas tidak hanya berdasarkan tingkat kritis, tetapi juga mempertimbangkan risiko akses, kebutuhan logistik, dan populasi terdampak.
7	Menggunakan Dijkstra untuk rute tercepat	Dijkstra cocok karena graph memiliki bobot jarak dan program perlu mencari rute dengan total jarak minimum.
8	Menggunakan Kruskal MST	Kruskal digunakan untuk merancang jaringan distribusi minimum agar semua lokasi tetap terhubung dengan total jarak sekecil mungkin.
9	Menambahkan fitur simulasi jalan rusak	Fitur ini dibuat untuk menunjukkan kondisi bencana susulan atau jalan tertutup yang mempengaruhi rute distribusi.

10	Menambahkan fitur cek jaringan	Fitur ini membantu mendeteksi apakah graph masih terhubung atau sudah terpecah menjadi beberapa komponen.
11	Menyimpan tambah dan update data ke CSV	Data yang ditambahkan atau diperbarui tidak hilang setelah program ditutup.
12	Menambahkan validasi input	Validasi dibuat agar program lebih aman saat demo dan tidak mudah error karena input salah.
13	Mencegah duplicate edge	Karena graph bersifat undirected, jalur A-B dan B-A dianggap sama sehingga duplikasi harus dicegah.

7. Tracing Manual

Tracing 1: Prioritas Bantuan dengan Min-Heap

Fitur ini menentukan lokasi yang paling prioritas menerima bantuan.

Rumus skor prioritas:

$$\text{Skor} = (\text{Kritis} \times 100) + (\text{Risiko} \times 40) + (\text{Kebutuhan} \times 5) + (\text{Populasi} / 100)$$

Contoh perhitungan:

Posko Porong

`criticalLevel = 5`

`riskLevel = 5`

`logisticsNeeded = 35.0`

`population = 7500`

$$\text{Skor} = (5 \times 100) + (5 \times 40) + (35.0 \times 5) + (7500 / 100)$$

$$\text{Skor} = 500 + 200 + 175 + 75$$

$$\text{Skor} = 950$$

Data dengan skor lebih tinggi memiliki prioritas lebih tinggi.

Contoh hasil prioritas:

1. Posko Porong
2. Posko Tanggulangin
3. Desa Renokenongo
4. Desa Lajuk
5. Desa Gempolsari

Alurnya:

1. Program membaca semua lokasi.
2. Lokasi bertipe gudang tidak dimasukkan ke antrean bantuan.
3. Setiap lokasi dihitung skor prioritasnya.
4. Data dimasukkan ke Min-Heap.
5. Program mengambil 5 lokasi dengan prioritas tertinggi.
6. Lokasi tersebut ditampilkan sebagai prioritas pengiriman bantuan.

Tracing 2: Dijkstra dari G01 ke D01

Tujuan: mencari rute tercepat dari G01 ke D01.

Data awal:

G01 = Gudang Pusat Surabaya

D01 = Desa Kedungcangkring

Langkah tracing:

1. Mulai dari G01.

Jarak awal:

G01 = 0

2. semua node lain = tak hingga

Tetangga dari G01:

P01 = 15.2 km

3. P05 = 18.5 km

4. Node dengan jarak terkecil berikutnya adalah P01.

5. Dari P01, program mengecek beberapa tetangga, salah satunya P02.

Jarak ke P02 melalui P01:

G01 -> P01 -> P02

6. $15.2 + 6.7 = 21.9$ km

7. Node berikutnya yang mengarah ke tujuan adalah P02.

8. Dari P02, program mengecek D01.

Jarak ke D01:

G01 -> P01 -> P02 -> D01

$$9. 15.2 + 6.7 + 2.1 = 24.0 \text{ km}$$

Hasil akhir:

Rute:

Gudang Pusat Surabaya -> Posko Sidoarjo Kota -> Posko Tanggulangin -> Desa Kedungcangkring

Total jarak:

24.0 km

Tracing 3: Kruskal MST

Tujuan: membuat jaringan distribusi minimum.

Alur Kruskal:

1. Program mengambil semua edge aktif.
2. Edge diurutkan dari jarak paling kecil ke paling besar.
3. Program memilih edge terkecil satu per satu.
4. Jika edge tidak membentuk siklus, edge dimasukkan ke MST.
5. Jika edge membentuk siklus, edge dilewati.
6. Proses berhenti ketika semua lokasi sudah terhubung.

Contoh edge kecil yang dipilih lebih awal:

Desa Besuki -- Desa Renokenongo == 1.2 km

Desa Besuki -- Posko Porong == 1.5 km

Desa Kalidawir -- Desa Ketapang == 1.5 km

Desa Kedungcangkring -- Desa Pejarakan == 1.8 km

Hasil akhir:

Total Jarak Jaringan: 78.6 km

Artinya, jaringan minimum dapat menghubungkan semua lokasi dengan total jarak 78.6 km

Tracing 4: Simulasi Jalan Rusak

Contoh simulasi:

Jalur G01 - P01 dibuat rusak

Alurnya:

1. User memilih menu simulasi jalan rusak.
2. User memasukkan lokasi pertama: G01.
3. User memasukkan lokasi kedua: P01.
4. User memilih status 2. Rusak.
5. Program mencari edge G01-P01 dan P01-G01.
6. Status edge diubah menjadi tidak aktif.
7. Saat jaringan ditampilkan ulang, jalur tersebut muncul sebagai rusak.

Output:

Jalur G01 - P01 statusnya menjadi: RUSAK/TERTUTUP

Dampaknya:

- Dijkstra tidak akan memakai jalur tersebut.
- Jika masih ada jalur lain, program mencari rute alternatif.
- Jika tidak ada jalur lain, program menampilkan bahwa rute tidak tersedia.

Tracing 5: Cek Konektivitas Jaringan

Tujuan: mengecek apakah semua lokasi masih tersambung melalui jalan aktif.

Alurnya:

1. Program memilih salah satu node awal.
2. Program menelusuri semua edge yang masih aktif.
3. Semua node yang bisa dicapai dimasukkan ke komponen yang sama.

4. Jika masih ada node yang belum dikunjungi, berarti ada komponen baru.
5. Jika jumlah komponen lebih dari 1, graph dianggap terputus.

Contoh output saat graph masih terhubung:

Status: TERHUBUNG

Semua lokasi masih dapat dijangkau melalui jalan aktif.

Total lokasi dalam jaringan aktif: 25

Contoh output saat graph terputus:

Status: TERPUTUS

Jaringan aktif terpecah menjadi 2 komponen.

Komponen 1 (... lokasi): ...

Komponen 2 (1 lokasi): PX2-Posko Terisolasi

8. Screenshot Hasil Program

```
Memuat dataset buatan sendiri...
Dataset berhasil dimuat (25 node).

=====
DISASTER RELIEF DISTRIBUTION NETWORK - FP 2026
=====
1. Tampilkan Semua Lokasi Posko & Gudang
2. Cari Lokasi Berdasarkan ID>Nama
3. Tambah Data Posko & Kebutuhan
4. Update Kebutuhan Logistik Lokasi
5. Tampilkan Jaringan Peta Saat Ini
6. Prioritaskan Pengiriman Bantuan (Min-Heap)
7. Cari Rute Tercepat Pengiriman (Dijkstra)
8. Rancang Jaringan Distribusi Minimum (Kruskal MST)
9. Simulasi Jalan Rusak / Bencana Susulan
10. Cek Konektivitas Jaringan
11. Keluar
Pilih menu: Terima kasih telah menggunakan sistem ini.
```

Tampilan Menu Utama

```

=== Daftar Lokasi ===
D01 | Desa Kedungcangkring | Tipe: Desa | Populasi: 850 | Tingkat Kritis: 5 | Kebutuhan: 3.5 Ton | Risiko: 5
D02 | Desa Pejarakan | Tipe: Desa | Populasi: 920 | Tingkat Kritis: 4 | Kebutuhan: 4.0 Ton | Risiko: 4
D03 | Desa Besuki | Tipe: Desa | Populasi: 1200 | Tingkat Kritis: 5 | Kebutuhan: 6.5 Ton | Risiko: 5
D04 | Desa Renokenongo | Tipe: Desa | Populasi: 1500 | Tingkat Kritis: 5 | Kebutuhan: 8.0 Ton | Risiko: 5
D05 | Desa Glagaharum | Tipe: Desa | Populasi: 600 | Tingkat Kritis: 3 | Kebutuhan: 2.0 Ton | Risiko: 3
D06 | Desa Ketapang | Tipe: Desa | Populasi: 1100 | Tingkat Kritis: 4 | Kebutuhan: 5.0 Ton | Risiko: 4
D07 | Desa Kalidawir | Tipe: Desa | Populasi: 950 | Tingkat Kritis: 4 | Kebutuhan: 4.5 Ton | Risiko: 4
D08 | Desa Gempolsari | Tipe: Desa | Populasi: 1300 | Tingkat Kritis: 5 | Kebutuhan: 7.0 Ton | Risiko: 5
D09 | Desa Sentul | Tipe: Desa | Populasi: 780 | Tingkat Kritis: 3 | Kebutuhan: 2.5 Ton | Risiko: 3
D10 | Desa Pamotan | Tipe: Desa | Populasi: 890 | Tingkat Kritis: 3 | Kebutuhan: 3.0 Ton | Risiko: 3
D11 | Desa Wunut | Tipe: Desa | Populasi: 1050 | Tingkat Kritis: 4 | Kebutuhan: 4.8 Ton | Risiko: 4
D12 | Desa Lajuk | Tipe: Desa | Populasi: 1400 | Tingkat Kritis: 5 | Kebutuhan: 7.5 Ton | Risiko: 5
D13 | Desa Kebonagung | Tipe: Desa | Populasi: 920 | Tingkat Kritis: 3 | Kebutuhan: 3.2 Ton | Risiko: 3
D14 | Desa Trompoasri | Tipe: Desa | Populasi: 1150 | Tingkat Kritis: 4 | Kebutuhan: 5.5 Ton | Risiko: 4
D15 | Desa Kupang | Tipe: Desa | Populasi: 1250 | Tingkat Kritis: 5 | Kebutuhan: 6.8 Ton | Risiko: 5
D16 | Desa Semambung | Tipe: Desa | Populasi: 800 | Tingkat Kritis: 2 | Kebutuhan: 1.5 Ton | Risiko: 2
D17 | Desa Kebakalan | Tipe: Desa | Populasi: 700 | Tingkat Kritis: 2 | Kebutuhan: 1.0 Ton | Risiko: 2
D18 | Desa Permisan | Tipe: Desa | Populasi: 1600 | Tingkat Kritis: 4 | Kebutuhan: 8.5 Ton | Risiko: 4
D19 | Desa Balongdowo | Tipe: Desa | Populasi: 900 | Tingkat Kritis: 3 | Kebutuhan: 3.0 Ton | Risiko: 3
G01 | Gudang Pusat Surabaya | Tipe: Gudang | Populasi: 0 | Tingkat Kritis: 1 | Kebutuhan: 0.0 Ton | Risiko: 1
P01 | Posko Sidoarjo Kota | Tipe: Posko | Populasi: 5000 | Tingkat Kritis: 4 | Kebutuhan: 15.5 Ton | Risiko: 3
P02 | Posko Tanggulangin | Tipe: Posko | Populasi: 3200 | Tingkat Kritis: 5 | Kebutuhan: 20.0 Ton | Risiko: 5
P03 | Posko Porong | Tipe: Posko | Populasi: 7500 | Tingkat Kritis: 5 | Kebutuhan: 35.0 Ton | Risiko: 5
P04 | Posko Jabon | Tipe: Posko | Populasi: 4100 | Tingkat Kritis: 3 | Kebutuhan: 10.0 Ton | Risiko: 3
P05 | Posko Candi | Tipe: Posko | Populasi: 2800 | Tingkat Kritis: 4 | Kebutuhan: 12.5 Ton | Risiko: 3

```

Tampilan Daftar Lokasi

11. Ketuan

Pilih menu: --- Jaringan Distribusi Bencana ---

Desa Kedungcangkring terhubung ke: Posko Tanggulangin(2.1km) Desa Pejarakan(1.8km) Desa Kebonagung(2.6km)
Desa Besuki terhubung ke: Posko Porong(1.5km) Desa Pejarakan(3.0km) Desa Renokenongo(1.2km)
Desa Pejarakan terhubung ke: Posko Tanggulangin(3.5km) Desa Kedungcangkring(1.8km) Desa Besuki(3.0km)
Desa Glagaharum terhubung ke: Posko Porong(4.0km) Desa Renokenongo(2.4km) Desa Gempolsari(3.9km)
Gudang Pusat Surabaya terhubung ke: Posko Sidoarjo Kota(15.2km) Posko Candi(18.5km)
Desa Renokenongo terhubung ke: Posko Porong(2.8km) Desa Besuki(1.2km) Desa Glagaharum(2.4km)
Desa Kalidawir terhubung ke: Posko Jabon(2.5km) Desa Ketapang(1.5km) Desa Trompoasri(2.2km)
Desa Ketapang terhubung ke: Posko Jabon(3.2km) Desa Kalidawir(1.5km)
Desa Sentul terhubung ke: Posko Candi(3.1km) Desa Pamotan(1.9km)
Desa Gempolsari terhubung ke: Posko Porong(5.5km) Desa Lajuk(4.1km) Desa Glagaharum(3.9km) Desa Kupang(4.5km)
Posko Sidoarjo Kota terhubung ke: Gudang Pusat Surabaya(15.2km) Posko Candi(4.3km) Posko Tanggulangin(6.7km)
Posko Porong terhubung ke: Posko Tanggulangin(3.8km) Posko Jabon(8.2km) Desa Besuki(1.5km) Desa Renokenongo(2.4km)
Posko Tanggulangin terhubung ke: Posko Sidoarjo Kota(6.7km) Posko Candi(5.1km) Posko Porong(3.8km) Desa Kedungcangkring(2.1km)
Posko Candi terhubung ke: Gudang Pusat Surabaya(18.5km) Posko Sidoarjo Kota(4.3km) Posko Tanggulangin(5.1km)
Posko Jabon terhubung ke: Posko Porong(8.2km) Desa Ketapang(3.2km) Desa Kalidawir(2.5km) Desa Trompoasri(3.7km)
Desa Pamotan terhubung ke: Posko Candi(4.6km) Desa Sentul(1.9km) Desa Balongdowo(2.7km)
Desa Lajuk terhubung ke: Posko Sidoarjo Kota(6.0km) Desa Gempolsari(4.1km) Desa Wunut(2.5km)
Desa Wunut terhubung ke: Posko Sidoarjo Kota(5.2km) Desa Lajuk(2.5km) Desa Semambung(3.3km)
Desa Trompoasri terhubung ke: Posko Jabon(3.7km) Desa Kalidawir(2.2km) Desa Kupang(3.6km)
Desa Kebonagung terhubung ke: Posko Tanggulangin(4.2km) Desa Kebakalan(2.0km) Desa Kedungcangkring(2.6km)
Desa Semambung terhubung ke: Posko Sidoarjo Kota(2.9km) Desa Wunut(3.3km)
Desa Kupang terhubung ke: Posko Jabon(5.1km) Desa Trompoasri(3.6km) Desa Gempolsari(4.5km)
Desa Permisan terhubung ke: Posko Candi(4.8km) Desa Balongdowo(3.8km)
Desa Kebakalan terhubung ke: Posko Tanggulangin(3.4km) Desa Kebonagung(2.0km)
Desa Balongdowo terhubung ke: Posko Candi(5.5km) Desa Pamotan(2.7km) Desa Permisan(3.8km)

Tampilan Jaringan Distribusi

Pilih menu:

=== Prioritas Pengiriman Bantuan Teratas (Min-Heap) ===

Skor = (Kritis x 100) + (Risiko x 40) + (Kebutuhan x 5) + (Populasi / 100)

1. Posko Porong (Skor: 950.0, Kritis: 5, Risiko: 5, Kebutuhan: 35.0 Ton, Populasi: 7500)
2. Posko Tanggulangin (Skor: 832.0, Kritis: 5, Risiko: 5, Kebutuhan: 20.0 Ton, Populasi: 3200)
3. Desa Renokenongo (Skor: 755.0, Kritis: 5, Risiko: 5, Kebutuhan: 8.0 Ton, Populasi: 1500)
4. Desa Lajuk (Skor: 751.5, Kritis: 5, Risiko: 5, Kebutuhan: 7.5 Ton, Populasi: 1400)
5. Desa Gempolsari (Skor: 748.0, Kritis: 5, Risiko: 5, Kebutuhan: 7.0 Ton, Populasi: 1300)

=====

DISASTER RELIEF DISTRIBUTION NETWORK - FP 2026

=====

1. Tampilkan Semua Lokasi Posko & Gudang
2. Cari Lokasi Berdasarkan ID/Nama
3. Tambah Data Posko & Kebutuhan
4. Update Kebutuhan Logistik Lokasi
5. Tampilkan Jaringan Peta Saat Ini
6. Prioritaskan Pengiriman Bantuan (Min-Heap)
7. Cari Rute Tercepat Pengiriman (Dijkstra)
8. Rancang Jaringan Distribusi Minimum (Kruskal MST)
9. Simulasi Jalan Rusak / Bencana Susulan
10. Cek Konektivitas Jaringan
11. Keluar

Pilih menu: Terima kasih telah menggunakan sistem ini.

Tampilan Min-Heap (Prioritas Pengiriman Bantuan Teratas)

Pilih menu:

Daftar ID Lokasi:

D01 - Desa Kedungcangkring
D02 - Desa Pejarakan
D03 - Desa Besuki
D04 - Desa Renokenongo
D05 - Desa Glagaharum
D06 - Desa Ketapang
D07 - Desa Kalidawir
D08 - Desa Gempolsari
D09 - Desa Sentul
D10 - Desa Pamotan
D11 - Desa Wunut
D12 - Desa Lajuk
D13 - Desa Kebonagung
D14 - Desa Trompoasri
D15 - Desa Kupang
D16 - Desa Semambung
D17 - Desa Kebakalan
D18 - Desa Permisan
D19 - Desa Balongdowo
G01 - Gudang Pusat Surabaya
P01 - Posko Sidoarjo Kota
P02 - Posko Tanggulangin
P03 - Posko Porong
P04 - Posko Jabon
P05 - Posko Candi

Masukkan ID lokasi asal: Masukkan ID lokasi tujuan:

=== Rute Tercepat ===

Dari: Gudang Pusat Surabaya

Ke: Desa Kedungcangkring

Total Jarak: 24.0 km

Jalur: Gudang Pusat Surabaya -> Posko Sidoarjo Kota -> Posko Tanggulangin -> Desa Kedungcangkring

Tampilan Dijkstra (Memilih Rute Tercepat)

Pilih menu:
=== Minimum Spanning Tree (Jaringan Distribusi Minimum) ===
Jalur yang dipertahankan untuk menghemat biaya operasional:
Desa Besuki -- Desa Renokenongo == 1.2 km
Desa Besuki -- Posko Porong == 1.5 km
Desa Kalidawir -- Desa Ketapang == 1.5 km
Desa Kedungcangkring -- Desa Pejarakan == 1.8 km
Desa Sentul -- Desa Pamotan == 1.9 km
Desa Kebonagung -- Desa Kebakalan == 2.0 km
Desa Kedungcangkring -- Posko Tanggulangin == 2.1 km
Desa Kalidawir -- Desa Trompoasri == 2.2 km
Desa Glagaharum -- Desa Renokenongo == 2.4 km
Desa Kalidawir -- Posko Jabon == 2.5 km
Desa Lajuk -- Desa Wunut == 2.5 km
Desa Kedungcangkring -- Desa Kebonagung == 2.6 km
Desa Pamotan -- Desa Balongdowo == 2.7 km
Posko Sidoarjo Kota -- Desa Semambung == 2.9 km
Desa Besuki -- Desa Pejarakan == 3.0 km
Desa Sentul -- Posko Candi == 3.1 km
Desa Wunut -- Desa Semambung == 3.3 km
Desa Trompoasri -- Desa Kupang == 3.6 km
Desa Permisan -- Desa Balongdowo == 3.8 km
Desa Glagaharum -- Desa Gempolsari == 3.9 km
Desa Gempolsari -- Desa Lajuk == 4.1 km
Posko Sidoarjo Kota -- Posko Candi == 4.3 km
Desa Gempolsari -- Desa Kupang == 4.5 km
Gudang Pusat Surabaya -- Posko Sidoarjo Kota == 15.2 km
Total Jarak Jaringan: 78.6 km

Tampilan Minimum Spanning Tree (Kruskal MST)

Pilih menu:

Daftar ID Lokasi:

D01 - Desa Kedungcangkring

D02 - Desa Pejarakan

D03 - Desa Besuki

D04 - Desa Renokenongo

D05 - Desa Glagaharum

D06 - Desa Ketapang

D07 - Desa Kalidawir

D08 - Desa Gempolsari

D09 - Desa Sentul

D10 - Desa Pamotan

D11 - Desa Wunut

D12 - Desa Lajuk

D13 - Desa Kebonagung

D14 - Desa Trompoasri

D15 - Desa Kupang

D16 - Desa Semambung

D17 - Desa Kebakalan

D18 - Desa Permisian

D19 - Desa Balongdowo

G01 - Gudang Pusat Surabaya

P01 - Posko Sidoarjo Kota

P02 - Posko Tanggulangin

P03 - Posko Porong

P04 - Posko Jabon

P05 - Posko Candi

Masukkan ID lokasi 1: Masukkan ID lokasi 2: Pilih status jalan:

1. Aktif

2. Rusak

Pilihan status: Jalur G01 - P01 statusnya menjadi: RUSAK/TERTUTUP

Desa Glagaharum terhubung ke: Posko Porong(4.0km) Desa Renokenongo(2.4km) Desa Gempolsari(3.9km)

Gudang Pusat Surabaya terhubung ke: [Posko Sidoarjo Kota - RUSAK] Posko Candi(18.5km)

Desa Renokenongo terhubung ke: Posko Porong(2.8km) Desa Besuki(1.2km) Desa Glagaharum(2.4km)

Tampilan Simulasi Jalan Rusak

Pilih menu:

=== Cek Konektivitas Jaringan ===

Status: TERHUBUNG

Semua lokasi masih dapat dijangkau melalui jalan aktif.

Total lokasi dalam jaringan aktif: 25

Tampilan Konektivitas Jaringan

Pilih menu:
=== Update Kebutuhan Logistik Lokasi ===

Daftar ID Lokasi:
D01 - Desa Kedungcangkring
D02 - Desa Pejarakan
D03 - Desa Besuki
D04 - Desa Renokenongo
D05 - Desa Glagaharum
D06 - Desa Ketapang
D07 - Desa Kalidawir
D08 - Desa Gempolsari
D09 - Desa Sentul
D10 - Desa Pamotan
D11 - Desa Wunut
D12 - Desa Lajuk
D13 - Desa Kebonagung
D14 - Desa Trompoasri
D15 - Desa Kupang
D16 - Desa Semambung
D17 - Desa Kebakalan
D18 - Desa Permisian
D19 - Desa Balongdowo
G01 - Gudang Pusat Surabaya
P01 - Posko Sidoarjo Kota
P02 - Posko Tanggulangin
P03 - Posko Porong
P04 - Posko Jabon
P05 - Posko Candi

Masukkan ID lokasi yang akan diupdate: P01 | Posko Sidoarjo Kota | Tipe: Posko | Populasi: 5000 | Tingkat Kritis: 4 | Kebutuhan: 15.5 Ton | Risiko: 3
Kebutuhan logistik baru (ton): Kebutuhan logistik P01 berhasil diupdate dari 15.5 Ton menjadi 22.5 Ton dan disimpan ke data/nodes.csv.

Tampilan Update Kebutuhan Logistik

9. Analisis Kompleksitas

Operasi	Struktur/Algoritma	Kompleksitas	Alasan
Insert prioritas	Min-Heap	$O(\log n)$	Elemen perlu dinaikkan dengan heapifyUp
Extract prioritas	Min-Heap	$O(\log n)$	Root diganti elemen terakhir lalu heapifyDown
Cari rute tercepat	Dijkstra + MinHeap	$O((V + E) \log V)$	Setiap edge dapat direlaksasi dan node diproses melalui heap
Jaringan minimum	Kruskal MST	$O(E \log E)$	Edge perlu diurutkan berdasarkan bobot
Cek konektivitas	BFS	$O(V + E)$	Semua vertex dan edge aktif dapat dikunjungi

Keterangan:
V : jumlah vertex atau lokasi.
E : jumlah edge atau ruas jalan.

Berdasarkan Kompleksitas Waktu diatas Min-Heap memungkinkan pengelolaan prioritas dilakukan dengan cepat, Dijkstra mampu menemukan rute tercepat secara optimal, sedangkan Kruskal menghasilkan jaringan distribusi minimum dengan biaya yang efisien.

10. What-if Analysis

1. Skenario Jalan Rusak

Pada skenario ini, salah satu jalan distribusi dibuat rusak atau tidak dapat dilewati. Contohnya adalah jalur antara **G01** yaitu Gudang Pusat Surabaya dan **P01** yaitu Posko Sidoarjo Kota.

Ketika user memilih fitur simulasi jalan rusak, program akan mengubah status edge **G01 - P01** menjadi tidak aktif. Setelah itu, jalur tersebut tidak akan digunakan lagi oleh algoritma Dijkstra ketika mencari rute tercepat.

Dampak dari skenario ini adalah:

- Jalur rusak tetap ditampilkan pada graph, tetapi diberi status rusak.
- Dijkstra akan mengabaikan jalur yang rusak.
- Jika masih ada rute alternatif, program akan mencari jalur lain.
- Jika tidak ada rute alternatif, program akan menampilkan bahwa rute tidak tersedia.

Dengan skenario ini, sistem dapat menunjukkan bagaimana jaringan distribusi bantuan beradaptasi ketika terjadi bencana susulan atau kerusakan jalan.

2. Skenario Graph Terputus

Pada skenario ini, jaringan distribusi tidak lagi terhubung sepenuhnya. Hal ini dapat terjadi jika ada lokasi baru yang belum dihubungkan ke jaringan, atau terlalu banyak jalan yang rusak sehingga suatu lokasi menjadi terisolasi.

Untuk menangani kondisi ini, program menyediakan fitur cek konektivitas jaringan. Fitur ini akan menelusuri graph berdasarkan edge yang masih aktif. Jika semua lokasi masih dapat dijangkau, maka program menampilkan status **TERHUBUNG**. Jika ada lokasi yang tidak dapat dijangkau, maka program menampilkan status **TERPUTUS**.

Contoh output ketika graph terputus:

Status: TERPUTUS

Jaringan aktif terpecah menjadi 2 komponen.

Dampak dari skenario ini adalah:

- Sistem dapat mengetahui apakah seluruh lokasi masih berada dalam satu jaringan distribusi.
- Jika graph terputus, program menampilkan jumlah komponen jaringan.
- Lokasi yang berada pada komponen berbeda kemungkinan tidak dapat dijangkau melalui rute aktif.
- Dijkstra tidak akan menemukan rute jika lokasi asal dan tujuan berada pada komponen yang berbeda.

Skenario ini penting karena dalam kondisi bencana, ada kemungkinan suatu desa atau posko tidak dapat dijangkau akibat kerusakan infrastruktur.

3. Skenario Kebutuhan Logistik Berubah

Pada skenario ini, kebutuhan logistik suatu lokasi berubah setelah data awal dimasukkan. Contohnya, kebutuhan logistik Posko Sidoarjo Kota meningkat dari **15.5 Ton** menjadi **22.5 Ton**.

Program menyediakan fitur update kebutuhan logistik. Ketika nilai kebutuhan logistik diperbarui, data tersebut akan disimpan kembali ke file **nodes.csv**, sehingga perubahan tetap ada meskipun program ditutup dan dijalankan kembali.

Dampak dari skenario ini adalah:

- Data kebutuhan logistik menjadi lebih sesuai dengan kondisi terbaru.
- Skor prioritas bantuan dapat berubah.
- Lokasi dengan kebutuhan logistik yang meningkat dapat naik dalam daftar prioritas.
- Min-Heap akan menghitung ulang prioritas berdasarkan data terbaru.

Skenario ini menunjukkan bahwa sistem tidak hanya menggunakan data statis, tetapi juga dapat menyesuaikan kondisi bantuan berdasarkan perubahan kebutuhan di lapangan.

4. Skenario Lokasi Baru Ditambahkan

Pada skenario ini, terdapat lokasi baru yang muncul setelah bencana terjadi. Misalnya terdapat posko darurat baru yang belum ada pada dataset awal. Kondisi seperti ini mungkin terjadi karena dalam situasi bencana, titik pengungsian atau posko bantuan dapat bertambah sesuai kebutuhan lapangan.

Program menyediakan fitur tambah data posko dan kebutuhan. Melalui fitur ini, user dapat memasukkan ID lokasi baru, nama lokasi, tipe lokasi, jumlah penduduk atau pengungsi, tingkat kritis, kebutuhan logistik, dan tingkat risiko akses.

Jika lokasi baru tersebut dihubungkan ke jaringan distribusi, maka program juga akan menambahkan jalur baru ke graph. Data lokasi baru disimpan ke file **nodes.csv**, sedangkan data jalur baru disimpan ke file **edges.csv**.

Dampak dari skenario ini adalah:

- Sistem dapat menyesuaikan dataset ketika ada posko atau desa baru.
- Lokasi baru dapat muncul pada daftar lokasi.
- Lokasi baru dapat dicari melalui fitur search.
- Jika sudah terhubung ke jaringan, lokasi baru dapat digunakan dalam pencarian rute Dijkstra.
- Lokasi baru juga dapat masuk ke daftar prioritas bantuan Min-Heap jika memiliki kebutuhan logistik.
- Data baru tetap tersimpan meskipun program ditutup dan dijalankan ulang.

Dengan skenario ini, sistem menjadi lebih fleksibel karena tidak hanya bergantung pada dataset awal. Sistem dapat menyesuaikan perubahan kondisi lapangan ketika ada titik distribusi bantuan baru.

5. Skenario Duplicate Edge Ditambahkan

Pada skenario ini, user mencoba menambahkan jalur distribusi yang sebenarnya sudah ada di dalam graph. Misalnya, jika sudah terdapat jalur antara **G01** dan **P01**, maka jalur **P01** ke **G01** juga dianggap sama karena graph yang digunakan bersifat undirected.

Program menangani kondisi ini dengan melakukan pengecekan sebelum edge baru dimasukkan ke adjacency list. Jika jalur tersebut sudah ada, maka program akan menolak penambahan edge baru.

Contoh kondisi:

Jalur **G01** - **P01** sudah ada.

User mencoba menambahkan jalur **P01** - **G01**.

Output yang diharapkan:

Jalur **P01** - **G01** sudah ada. Duplikasi edge dibatalkan.

Dampak dari skenario ini adalah:

- Graph tidak memiliki jalur duplikat.
- Adjacency list tetap bersih dan tidak menyimpan edge yang sama berkali-kali.
- Perhitungan Dijkstra dan Kruskal MST tetap lebih aman karena tidak ada data jalur ganda yang tidak diperlukan.
- Dataset **edges.csv** tidak diisi dengan relasi yang sama secara berulang.
- Struktur graph menjadi lebih konsisten.

Dengan skenario ini, sistem dapat menjaga kualitas data graph. Hal ini penting karena duplicate edge dapat membuat data menjadi tidak rapi dan dapat membingungkan saat analisis jaringan distribusi.